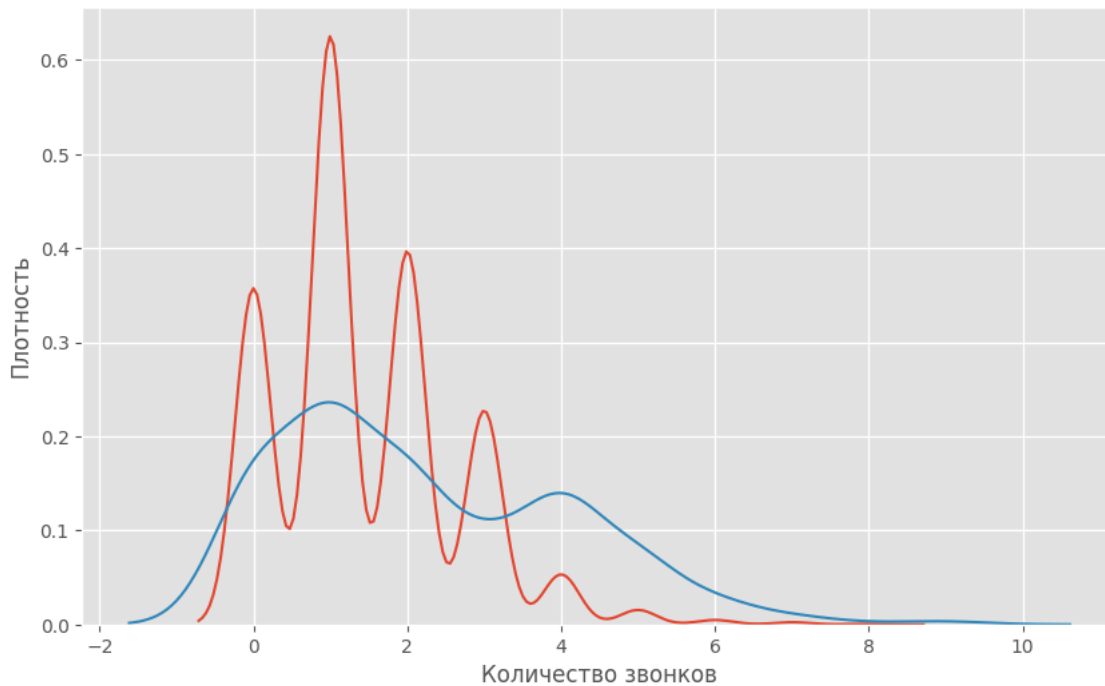


```

1 import pandas as pd # pandas для работы с таблицами
2 from matplotlib import pyplot as plt # графики
3 plt.style.use('ggplot') # стиль графиков
4 plt.rcParams['figure.figsize'] = 10, 6 # размер окна графиков
5 import seaborn as sns # для графиков
6 %matplotlib inline
7 URL='https://drive.google.com/uc?id=1U9LC5ECI2_Jvq4FrZNCOn3gCT7T5Y5DD'
8 telecom_data = pd.read_csv(URL) # загружаем данные в таблицу
9 # выбираем и отображаем звонки (столбец 'Customer service calls') лояльных клиентов (столбец ['Churn'])
10 fig = sns.kdeplot(telecom_data[telecom_data['Churn'] == False]['Customer service calls'])
11 # выбираем и отображаем звонки (столбец 'Customer service calls') нелояльных, бывших клиентов (столбец ['Churn'])
12 fig = sns.kdeplot(telecom_data[telecom_data['Churn'] == True]['Customer service calls'])
13 fig.set(xlabel='Количество звонков', ylabel='Плотность') # подписи осей
14 plt.show() # отображаем
15 #https://www.pvsm.ru/python/250970
16

```



```

1 import numpy as np #
2 def get_bootstrap_samples(data, n_samples, n_data):
3     # функция для генерации подвыборок с помощью бутстрэпа
4     # выбираем случайно n_samples на n_data индексов данных data
5     # randint автоматически обеспечивает повторения
6     indices = np.random.randint(0, len(data), (n_samples, n_data))
7     # отбираем по этим индексам данные
8     samples = data[indices]
9     return samples # возвращаем
10 def stat_intervals(stat, alpha):
11     # функция для интервальной оценки
12     # считаем границы интервала (перцентили) для заданного процента вероятности alpha
13     boundaries = np.percentile(stat, [100 * alpha / 2., 100 * (1 - alpha / 2.)])
14     return boundaries #
15 n_samples=1000; # число подвыборок
16 n_data=500; # размер подвыборки
17 # сохранение в отдельные numpy массивы данных по лояльным и уже бывшим клиентам
18 loyal_calls = telecom_data[telecom_data['Churn'] == False]['Customer service calls'].values
19 churn_calls = telecom_data[telecom_data['Churn'] == True]['Customer service calls'].values
20 # ставим seed для воспроизводимости результатов
21 np.random.seed(0)
22 # генерируем выборки с помощью бутстрэпа и сразу считаем по каждой из них среднее
23 loyal_mean_scores = [np.mean(sample) for sample in get_bootstrap_samples(loyal_calls, n_samples, n_data)]
24 churn_mean_scores = [np.mean(sample) for sample in get_bootstrap_samples(churn_calls, n_samples, n_data)]
25 # выводим интервальную оценку среднего
26 print("Service calls from loyal: mean interval", stat_intervals(loyal_mean_scores, 0.05))
27 print("Service calls from churn: mean interval", stat_intervals(churn_mean_scores, 0.05))
28 # просто среднее
29 print("Service calls from loyal: mean ", np.mean(loyal_calls))
30 print("Service calls from churn: mean ", np.mean(churn_calls))
31

```

```

Service calls from loyal: mean interval [1.35  1.556]
Service calls from churn: mean interval [2.06  2.3961]
Service calls from loyal: mean 1.4498245614035088

```

Service calls from churn: mean 2.229813664596273

```

1 import numpy as np #
2 import matplotlib.pyplot as plt #
3 from sklearn.ensemble import BaggingRegressor # беггинг
4 from sklearn.tree import DecisionTreeRegressor # дерево для регрессии
5 # Установки
6 n_repeat = 50 # Число итераций для генерации шума
7 n_train = 50 # число примеров обучения
8 n_test = 1000 # число примеров теста
9 noise = 0.1 # дисперсия (стандартное отклонение) шума
10 np.random.seed(0) # начало генератора случайных чисел
11 # проверяемые модели, дерево и ансамбль деревьев
12 # по умолчанию в ансамбле будет 10 деревьев.
13 estimators = [("Tree", DecisionTreeRegressor()),
14 ("Bagging(Tree)",
15 BaggingRegressor(DecisionTreeRegressor(),
16 n_estimators=10,max_samples=1.0))]
17 # число проверяемых моделей
18 n_estimators = len(estimators)
19 # Функция для генерации данных
20 def f(x):
21     x = x.ravel() #
22     # пример красивой функции
23     return np.exp(-x ** 2) + 1.5 * np.exp(-(x - 2) ** 2)
24 # функция для генерации и добавления шума в данные
25 def generate(n_samples, noise, n_repeat=1):
26     # n_samples - число примеров,
27     # noise - дисперсия шума
28     # n_repeat - число повторов
29     # входы в функцию - абциссы по оси OX, случайно выбранные
30     X = np.random.rand(n_samples) * 10 - 5 # некоторые случайные числа от -5 до 5
31     X = np.sort(X) # отсортированы
32     # Выходы функции
33     if n_repeat == 1: # если повтор один
34         # добавляем шум - случайные числа с нормальным распределением, с нулевым средним, с заданн
35         y = f(X) + np.random.normal(0.0, noise, n_samples) #
36     else:
37         y = np.zeros((n_samples, n_repeat)) # иначе сначала забиваем нулями
38         for i in range(n_repeat): # а потом в цикле по количеству повторов
39             # делаем несколько зашумленных функций
40             y[:, i] = f(X) + np.random.normal(0.0, noise, n_samples)
41     X = X.reshape((n_samples, 1)) # отформатируем входы в этом виде
42     return X, y # возвратим входы и выходы функции с шумом
43 X_train = [] # место под обучающие входы
44 y_train = [] # место под обучающие выходы
45 for i in range(n_repeat): # в цикле по количеству повторов
46     X, y = generate(n_samples=n_train, noise=noise) # генерируем данные с шумом
47     X_train.append(X) # добавляем их ко входам
48     y_train.append(y) # и выходам
49 # то же для теста
50 X_test, y_test = generate(n_samples=n_test, noise=noise, n_repeat=n_repeat)
51 plt.figure(figsize=(10, 8)) # полотно для рисования
52 # В цикле по количеству моделей
53 for n, (name, estimator) in enumerate(estimators): #
54     # вычисляем выходы
55     y_predict = np.zeros((n_test, n_repeat)) # место под выходы
56     #
57     for i in range(n_repeat): # в цикле по количеству повторов
58         estimator.fit(X_train[i], y_train[i]) # обучаем модель на своих данных
59         y_predict[:, i] = estimator.predict(X_test) # считаем выход модели
60 # Bias^2 + Variance + Noise вычисляем составляющие ошибки
61 y_error = np.zeros(n_test) # место под результаты теста
62 # средняя ошибка
63 for i in range(n_repeat): # в цикле по количеству повторов
64     for j in range(n_repeat): # еще раз в цикле по количеству повторов
65         # находим сумму квадратов разности истинного и рассчитанного выхода для всех комбинаций
66         y_error += (y_test[:, j] - y_predict[:, i]) ** 2
67 y_error /= (n_repeat * n_repeat) # делим на количество комбинаций - получаем среднее
68 y_noise = np.var(y_test, axis=1) # часть ошибки от шума (дисперсия).
69 # Примечание: т.к. дисперсия считается для отклонения от среднего значения, то нам не надо сам
70 # axis=1 показывает что мы ищем по дисперсию по повторам, а не по примерам.
71 y_bias = (f(X_test) - np.mean(y_predict, axis=1)) ** 2 # часть ошибки от смещения - квадрат от
72 y_var = np.var(y_predict, axis=1) # часть ошибки от дисперсии результатов
73 # выводим эти числа на экран
74 print("{0:13.15}: {1:.4f} (error) = {2:.4f} (bias^2) "
75 " + {3:.4f} (var) + {4:.4f} (noise)".format(name,
76 np.mean(y_error),
77 np.mean(y_bias),
78 np.mean(y_var),
79 np.mean(y_noise)))
80 # Рисуем картинку

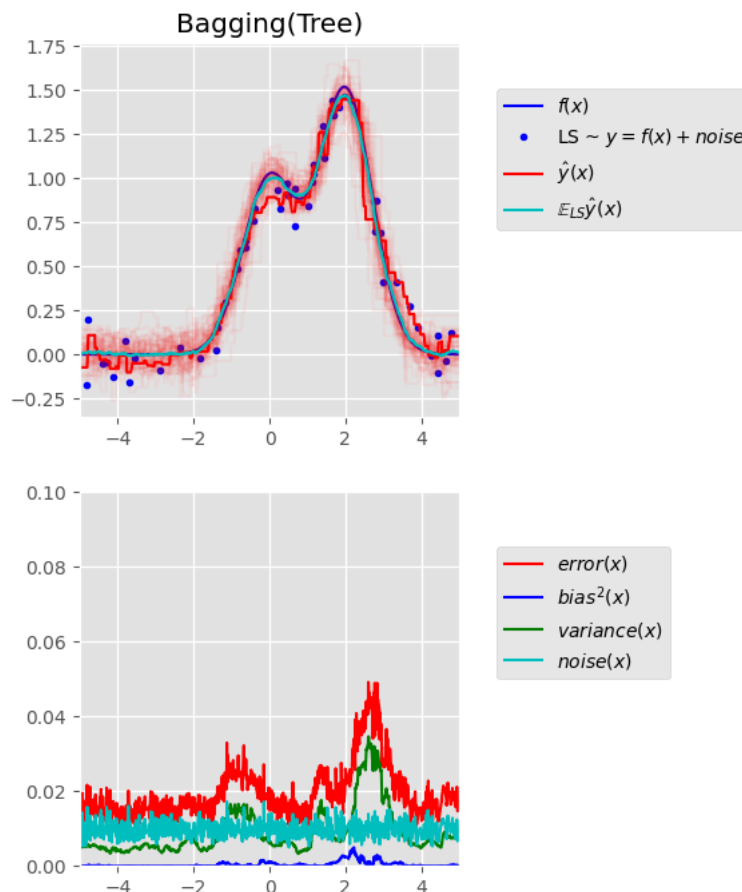
```

```

81 plt.subplot(2, n_estimators, n + 1) # свои подграфики каждой модели
82 plt.plot(X_test, f(X_test), "b", label="$f(x)$") # истинная функция
83 plt.plot(X_train[0], y_train[0], ".b", label="LS ~ $y = f(x)+noise$") # обучающие точки
84 #
85 for i in range(n_repeat): # в цикле по количеству повторов
86     if i == 0: # если это первый повтор рисуем результаты
87         plt.plot(X_test, y_predict[:, i], "r", label=r"$\hat{y}(x)$")
88     else: # другие повторы рисуем полупрозрачным цветом
89         plt.plot(X_test, y_predict[:, i], "r", alpha=0.05)
90 # рисуем среднее значение
91 plt.plot(X_test, np.mean(y_predict, axis=1), "c",
92 label=r"$\mathbb{E}_{LS} \hat{y}(x)$")
93 plt.xlim([-5, 5]) # область просмотра графика
94 plt.title(name) # название графика
95 # легенда графика на последней итерации
96 if n == n_estimators - 1:
97     plt.legend(loc=(1.1, .5)) #
98 # на других подграфиках рисуем:
99 plt.subplot(2, n_estimators, n_estimators + n + 1) #
100 plt.plot(X_test, y_error, "r", label="$error(x)$") # ошибку модели
101 plt.plot(X_test, y_bias, "b", label="$bias^2(x)$"), # квадрат смещения ошибки модели
102 plt.plot(X_test, y_var, "g", label="$variance(x)$"), # дисперсию ошибки модели
103 plt.plot(X_test, y_noise, "c", label="$noise(x)$") # дисперсию шума модели
104 #
105 plt.xlim([-5, 5]) # область просмотра графика по X
106 plt.ylim([0, 0.1]) # область просмотра графика по Y
107 if n == n_estimators - 1: # легенда
108     plt.legend(loc=(1.1, .5)) #
109 plt.subplots_adjust(right=.75) # немного сожмем графики настроив правую границу их, чтобы лучше см
110 plt.show() # отображаем графики
111

```

Bagging(Tree): 0.0196 (error) = 0.0004 (bias²) + 0.0092 (var) + 0.0098 (noise)



```

1 # Классификация с шумом
2 # функция для рисования разделяющей поверхности на плоскости
3 # Div_plate(clf, flip_level) принимает объект-классификатор и уровень шума (ошибок)
4 # создает двумерные данные, обучает классификатор, рисует разделяющую поверхность
5 from sklearn.datasets import make_classification # для генерации данных
6 from sklearn.model_selection import train_test_split # для разделения на обучение и тест
7 import numpy as np
8 import seaborn as sns # для простого отображения
9 import matplotlib.pyplot as plt #
10 # создаем данные
11 def Div_plate(clf, flip_level):
12

```

```

13 X,y = make_classification(n_samples=10000, # число примеров
14     n_features=2, # число признаков (атрибутов)
15     n_informative=2, # из них информативных
16     n_redundant=0, # из них не информативных
17     n_repeated=0, # из них повторяющихся
18     n_classes=2, # число классов
19     n_clusters_per_class=1, # число кластеров на класс
20     class_sep=2, # влияет на расстояние между кластерами
21     flip_y=flip_level, # доля ошибок (шума)
22     weights=[0.5,0.5], # пропорции числа данных в классах
23     random_state=17) #
24 # разделяем на обучающие и тестовые, случайно
25 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
26 # рисуем данные
27 plt.subplots();#
28 ax1=plt.gca();#
29 sns.scatterplot(x=X_train[:,0],y=X_train[:,0],hue=y_train,ax=ax1);# обучающие
30 sns.scatterplot(x=X_test[:,0],y=X_test[:,0],hue=y_test,ax=ax1,marker="+");# тестовые
31 ax1.set_title("Data With Noise");#
32 plt.show();#
33 # Создаем классификатор на основе ансамбля. Изменяйте параметры и смотрите как это влияет на о
34 clf.fit(X_train, y_train) # обучаем
35 y_pred = clf.predict(X_test) # проверяем на тестовых данных
36 score=clf.score(X_test, y_test) # считаем среднюю точность
37 print('Score=',score)
38 ind=y_test==y_pred; # индексы совпадений результата классификации и меток классов
39 plt.subplots();
40 ax2=plt.gca();
41 # рисуем "правильно" распознанные примеры
42 sns.scatterplot(x=X_test[ind,0],y=X_test[ind,1],hue=y_test[ind],ax=ax2);
43 # рисуем "неправильно" распознанные примеры
44 sns.scatterplot(x=X_test[~ind,0],y=X_test[~ind,1],hue=y_pred[~ind],ax=ax2,marker="+");
45 ax2.set_title("With Noise");
46 # считаем и рисуем разделяющую поверхность.
47 plot_step=0.01
48 x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1 # немного измененные минимальное и максима
49 y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1 # немного измененные минимальное и максима
50 # считаем прямоугольную сетку возможных значений этих атрибутов
51 xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step), #
52     np.arange(y_min, y_max, plot_step)) #
53 # считаем выход классификатора для всех примеров сетки
54 # не забыв что массивы данных нужно привести к требуемому размеру.
55 Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
56 Z = Z.reshape(xx.shape) # и преобразуем обратно в исходному размеру
57 # рисуем разделяющую поверхность
58 cs = plt.contourf(xx, yy, Z, levels=1, colors=['blue','orange'],alpha=0.3) # рисуем контурную
59 plt.show();
60

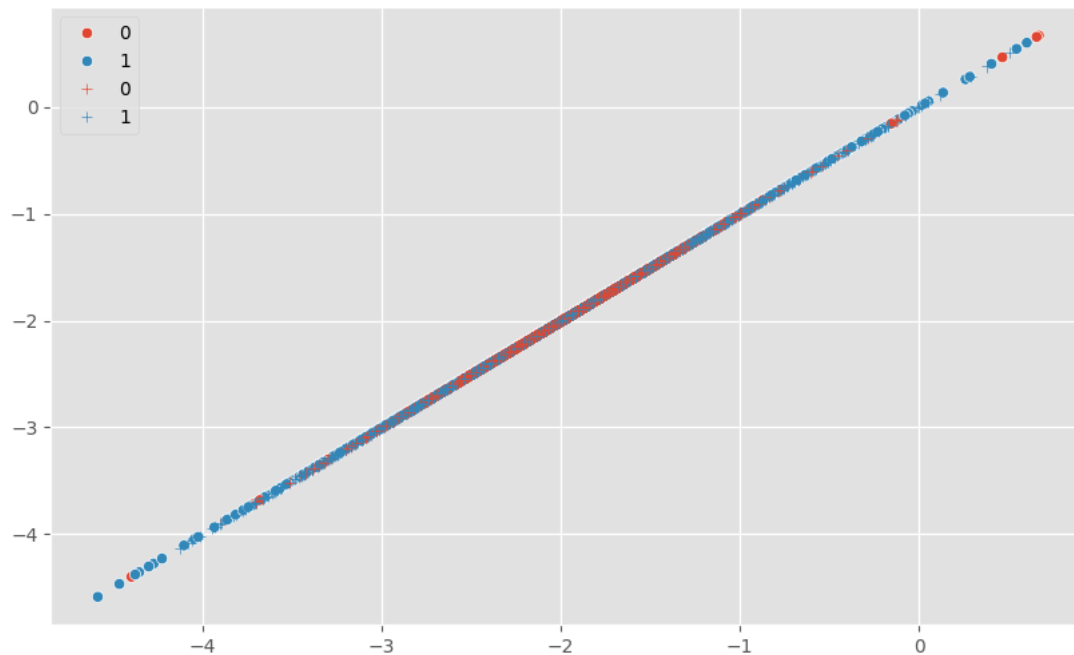
```

```

1 from sklearn.ensemble import BaggingClassifier #
2 from sklearn.tree import DecisionTreeClassifier # дерево для перепрессии
3 clf=BaggingClassifier(DecisionTreeClassifier(), n_estimators=10,max_samples=0.5)
4 flip_level=0.2
5 Div_plate(clf,flip_level)
6

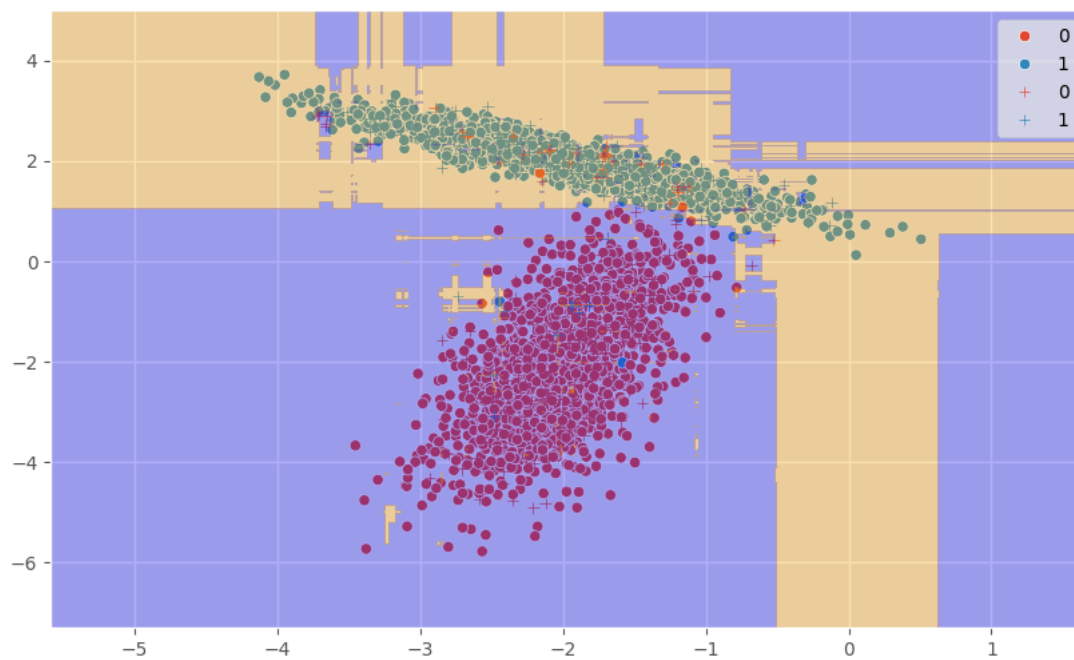
```

Data With Noise



Score= 0.883030303030303

With Noise



```

1 from sklearn.ensemble import BaggingClassifier #
2 from sklearn.linear_model import LogisticRegression # модель для регрессии
3 #from sklearn.linear_model import LinearRegression # модель для регрессии
4 clf=BaggingClassifier(LogisticRegression(), n_estimators=10,max_samples=0.5, max_features=0.5)
5 #clf=BaggingClassifier(LinearRegression(), n_estimators=10,max_samples=0.5, max_features=0.5)
6 flip_level=0.2
7 Div_plate(clf,flip_level)

```

